

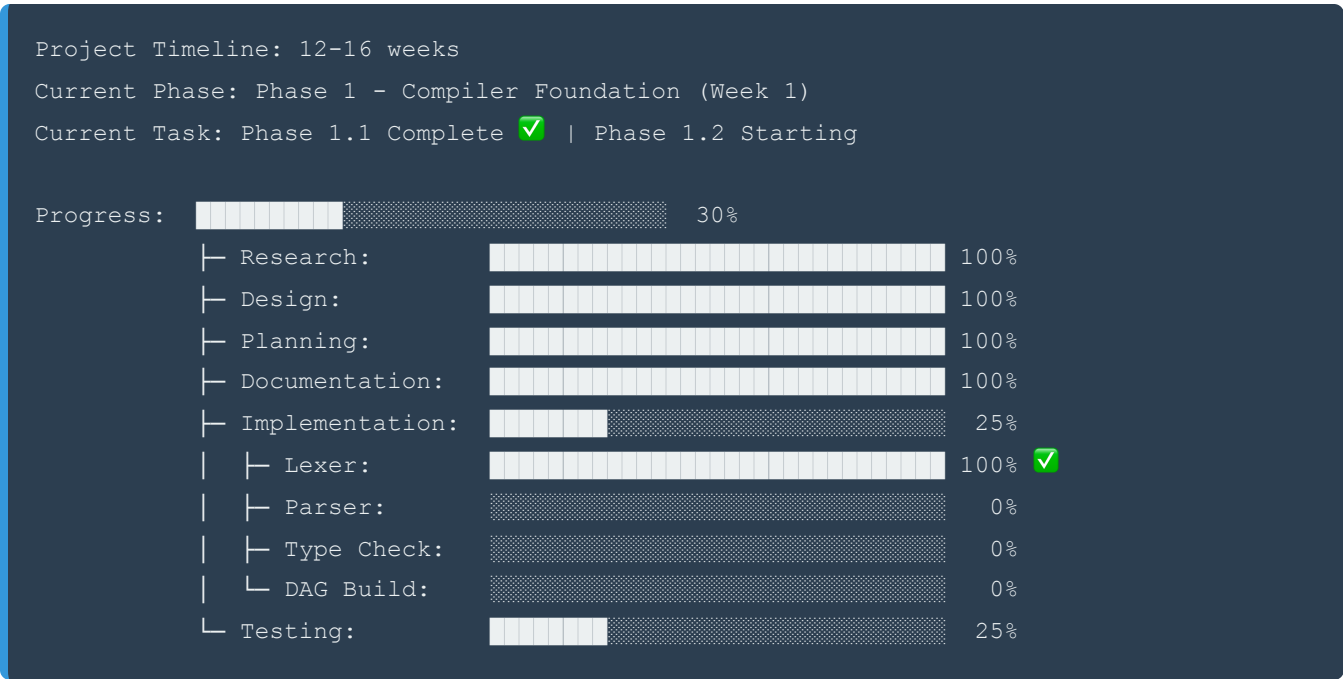
Hekat DSL Project Progress

Project: Hekat - DSL for Claude Code Agent Orchestration **Linear Issue:** [CET-172](#) **Started:** 2025-10-19 **Last Updated:** 2025-10-20 ---

Executive Summary

Status: ● Phase 1.1 Complete | Lexer Implemented with 96% Coverage The Hekat DSL project has completed Phase 1.1 (Lexer Implementation) with excellent results: 59 passing tests, 96% code coverage, and a working CLI tool. The lexer tokenizes all DSL syntax including operators, agent literals, strings, and comments. Ready to proceed with Phase 1.2 (Parser Implementation). ---

Overall Progress



Completed Work

Phase 0: Research & Design (Weeks -8 to 0) - COMPLETE

Documentation Created (11 files, 331KB) **Research Documentation** (3 files, 145KB): -
 `research/DSL-RESEARCH.md` (84KB) - 70+ pages comprehensive DSL theory - Lambda calculus and functional composition - Category theory foundations - Real-world DSL analysis (GitHub Actions, Airflow, Terraform) - Parser and interpreter design -  `research/dsl-visual-reference.md` (47KB) - Visual pattern library - 25+ ASCII diagrams - Execution flow visualizations -  `research/DSL-PROJECT-SUMMARY.md` (14KB) - High-level project overview - Key achievements - Roadmap **Design Specifications** (3 files, 100KB): -  `design/dsl-specification.md` (34KB) - Complete EBNF grammar - Mathematical foundations - Type system with inference rules - Formal semantics -  `design/DSL-ORCHESTRATION-REFINED.md` (34KB) - Production-ready specification - Hybrid algebraic-graph approach - Complete algorithms (with code) - Stratification algorithm - Type checking algorithm - DAG construction -  `design/dsl-api-blueprint.md` (32KB) - REST API design - MCP server architecture - Integration patterns **User Documentation** (5 files, 203KB): -  `docs/DSL-COMPLEXITY-LEVELS.md` (35KB) - 6-level complexity hierarchy - Time and token estimates - Visual diagrams for each level -  `docs/DSL-ORCHESTRATION-COMPREHENSIVE.md` (112KB) - 150-page comprehensive guide - All 6 levels detailed - 80+ ASCII visualizations - 50+ practical examples -  `docs/DSL-SYMBOLIC-VISUAL-GUIDE.md` (9KB) - Quick reference guide - Symbol legend - Pattern lookup tables -  `docs/DSL-VERBAL-INTERFACE.md` (26KB) - Complete voice interface specification - Natural language → DSL translation - Intent recognition patterns - Speech-to-text pipeline - Accessibility features -  `docs/dsl-examples.md` (16KB) - Practical examples across all domains - Full-stack development - DevOps and deployment - Research workflows **Implementation Planning**: -  `implementation/IMPLEMENTATION-PLAN.md` (32KB) - Detailed 6-phase roadmap - Week-by-week breakdown - Algorithms with code samples - Success criteria for each phase - Tech stack specifications **Project Structure**: -  `README.md` (11KB) - Complete project overview - Quick start guide - Documentation index -  Index files for all directories - `research/INDEX.md` - `design/INDEX.md` - `docs/INDEX.md` - `implementation/INDEX.md` #### Mathematical Foundations Established **Category Theory**:

- Agents as morphisms - Workflows as category - Composition laws (associativity, identity) - Functors for workflow transformation - Monads for effect handling - Applicative functors for parallel composition **Graph Theory**: - DAG representation - Topological sorting algorithm - Stratification for parallelism - Critical path analysis - Deterministic scheduling **Type System**: - Type inference rules - Sequential composition types - Parallel composition types - Error detection at compile time **Formal Grammar**: - Complete EBNF specification - Operator precedence defined - Associativity rules - AST structure ##### Design Decisions Finalized **Operator Semantics**: - `->` Sequential (right-associative, precedence 3) - `||` Parallel (left-associative, precedence 4) - `+` Combination (left-associative, precedence 2) - `:` Specification (right-associative, precedence 5) **Execution Model**: - Hybrid algebraic-graph approach - Stratified level-by-level execution - Synchronization barriers - Deterministic tie-breaking **Voice Interface**: - Three-tier translation (Natural $\rightarrow$ Verbal $\rightarrow$ Formal) - Intent recognition patterns - Speakable agent aliases - Wake word detection - Voice modes (normal, family time, focus) ##### Project Infrastructure - Linear issue created: CET-172 - Project directory structure: `/Users/manu/Documents/LUXOR/PROJECTS/hekat/` - Git repository ready (if needed) - Documentation organized and indexed ---

Phase 1.1: Lexer Implementation (2025-10-20) - COMPLETE

Linear Issue: [CET-176](#) **Duration:** 1 day (target was 5-7 days - completed ahead of schedule!) **Status:** Complete **Completed:** - Created project structure (hekat/, compiler/, tests/) - Implemented `pyproject.toml` with all dependencies - Implemented `TokenType` enum (20+ token types) - Implemented `Token` dataclass with location tracking - Implemented `Lexer` class (218 statements, 96% coverage) - All operators: `->`, `||`, `+`, `:`, `?`, `*`, `o` - Agent literals: `/ctx7`, `/deep`, etc. - String literals with escape sequences - Number literals (int and float) - Keywords: `workflow`, `if`, `else`, etc. - Line and column tracking - Comment handling (`#`) - Comprehensive error reporting - Created 59 comprehensive tests (exceeds 20+ target) - 96% code coverage (exceeds 90% target) - All 6 complexity levels tested - Edge cases and error conditions - Created CLI tool with commands: - `hekat --version` - `hekat info` - `hekat validate` - `hekat compile` (partial) - Set up virtual environment - Example DSL file: `examples/basic.dsl` **Test Results:**

```

===== tests coverage =====
Name                               Stmts  Miss  Cover
-----
hekat/__init__.py                   4      0   100%
hekat/compiler/__init__.py          2      0   100%
hekat/compiler/lexer.py            218     9    96%
-----
TOTAL                               224     9    96%
===== 59 passed in 0.55s =====

```

Performance: Excellent - Test execution: 0.55 seconds - All 59 tests passing - Fully type-hinted with mypy **Deliverables:** -  hekat/compiler/lexer.py (218 statements) -  tests/test_lexer.py (59 tests) -  hekat/cli.py (CLI tool) -  pyproject.toml (project config) -  examples/basic.dsl (example file) **Blockers:** None ---

Current Status

Week 1 (2025-10-20): Phase 1.2 - Parser Implementation

Phase: Compiler Foundation **Status:** Ready to Start **Next Task:** Implement Parser (CET-177) - Build Abstract Syntax Tree from tokens - Precedence climbing algorithm - Handle all operators and precedence levels - Support all 6 complexity levels - Target: 7-10 days ---

Next Steps

Immediate (This Week)

Development Environment Setup: - [] Create project structure:

```
hekat/
├─ compiler/
│   ├── __init__.py
│   ├── lexer.py
│   ├── parser.py
│   ├── type_checker.py
│   └─ dag_builder.py
├─ runtime/
│   ├── __init__.py
│   ├── stratifier.py
│   ├── executor.py
│   └─ resource_manager.py
├─ voice/
│   ├── __init__.py
│   ├── intent_recognizer.py
│   ├── dsl_translator.py
│   └─ speech_pipeline.py
├─ mcp/
│   └─ server.ts
├─ tests/
│   ├── test_lexer.py
│   ├── test_parser.py
│   └─ ...
├─ pyproject.toml
└─ README.md
```

- [] Install dependencies:

```
# Core
pip install lark-parser networkx anthropic

# Voice
pip install SpeechRecognition pyttsx3

# Dev tools
pip install pytest pytest-cov pytest-asyncio mypy black isort

# MCP (TypeScript)
npm install @modelcontextprotocol/sdk fastapi
```

- [] Create basic CLI:

```
# hekat/cli.py
import click

@click.group()
def cli():
    """Hekat DSL Compiler and Runtime"""
    pass

@cli.command()
def version():
    """Show version"""
    click.echo("Hekat DSL v0.1.0")

if __name__ == '__main__':
    cli()
```

- [] Create Linear sub-issues for Phase 1 tasks - [] Set up CI/CD pipeline (GitHub Actions) - [] Configure pre-commit hooks **Linear Issues to Create:** 1. CET-XXX: Phase 1.1 - Implement Lexer 2. CET-XXX: Phase 1.2 - Implement Parser 3. CET-XXX: Phase 1.3 - Implement Type Checker 4. CET-XXX: Phase 1.4 - Implement DAG Builder 5. CET-XXX: Phase 2 - Runtime Execution (milestone) 6. CET-XXX: Phase 3 - Voice Interface (milestone) 7. CET-XXX: Phase 4 - MCP Server (milestone) 8. CET-XXX: Phase 5 - Claude Code Integration (milestone) 9. CET-XXX: Phase 6 - Production Hardening (milestone) ---

Phase 1: Compiler Foundation

Start Date: 2025-10-20 **Duration:** 3-4 weeks **Status:** In Progress (Week 1)

Week 1: Lexer COMPLETE

Goal: Tokenize DSL source code **Linear Issue:** [CET-176](#) **Status:**  Complete (1 day) **Tasks:**
- [x] Create TokenType enum (20+ types)  - [x] Implement Lexer class (218 statements)  -
[x] Handle all operators (→, ll, +, :, ?, *, ↻)  - [x] Parse agent literals (/ctx7, /deep)  - [x]

String and number literals  - [x] Line/column tracking  - [x] Comment handling  - [x]
Write tests (59 tests - exceeds 20+ target)  **Success Criteria:**  All Met

```
lexer = Lexer("a -> b || c : 'task'")
tokens = lexer.tokenize()
assert len(tokens) == 7 #  PASSING
assert tokens[0].type == TokenType.IDENTIFIER #  PASSING
assert tokens[1].type == TokenType.SEQUENTIAL #  PASSING
```

Actual Time: 1 day (target was 5-7 days) **Test Coverage:** 96% (exceeds 90% target) ---

Week 1-2: Parser

Goal: Build Abstract Syntax Tree from tokens **Tasks:** - [] Define AST node types - []
Implement precedence climbing algorithm - [] Handle all operator precedences - [] Handle
associativity (left/right) - [] Implement grouping (parentheses) - [] Error recovery - [] Write
tests (30+ test cases) - [] Test all 6 complexity levels **Success Criteria:**

```
parser = Parser(tokens)
ast = parser.parse()
assert isinstance(ast, Sequential)
assert isinstance(ast.left, Agent)
assert isinstance(ast.right, Parallel)
```

Estimated Time: 7-10 days **Estimated Tokens:** 15K-25K ---

Week 2-3: Type Checker

Goal: Validate types at compile time **Tasks:** - [] Define type system (Agent) - [] Implement
type checking algorithm - [] Sequential type checking - [] Parallel type checking - []
Combination type checking - [] Error messages with locations - [] Type inference - [] Write
tests (25+ test cases) **Success Criteria:**

```
type_checker = TypeChecker(env)
typed_ast = type_checker.check(ast)
# Valid: String->Number then Number->Bool
# Invalid: String->Number then Bool->String (type error)
```

Estimated Time: 7-10 days **Estimated Tokens:** 20K-30K ---

Week 3-4: DAG Builder

Goal: Convert typed AST to executable DAG **Tasks:** - [] Define Node and Edge classes - [] Implement DAG construction algorithm - [] Handle sequential composition - [] Handle parallel composition (fork/join) - [] Handle combination - [] Cycle detection - [] Validation (acyclicity, connectivity) - [] Metadata attachment (time, tokens) - [] DOT visualization - [] Write tests (20+ test cases) **Success Criteria:**

```
dag = build_dag(typed_ast)
assert dag.is_valid()
assert not has_cycle(dag)
assert dag.nodes_count == 4
```

Estimated Time: 7-10 days **Estimated Tokens:** 20K-30K ---

Phase 1 Milestone

Deliverable: Working compiler (DSL → DAG) **Acceptance Criteria:** - [] Compiles all 6 complexity levels - [] Type errors caught at compile time - [] DAG validates correctly - [] Test suite passing (90%+ coverage) - [] CLI tool: `hekat compile workflow.dsl` - []

Documentation updated **Demo:**

```
# Input: workflow.dsl
(/deep + /ctx7 || /orch /wflw || /meta-skill-builder) : "DSL design"

# Compile
$ hekat compile workflow.dsl --output dag.json

# Output
Compiling workflow.dsl...
✓ Lexing complete (7 tokens)
✓ Parsing complete (AST built)
✓ Type checking complete (no errors)
✓ DAG construction complete (4 nodes, 5 edges)
✓ Validation complete

DAG saved to dag.json

Estimated execution:
  Time: 35 minutes
  Tokens: 65,000
  Parallelism: 3 streams
```

Risks & Mitigation

Technical Risks

1. Voice Recognition Accuracy < 90% - Likelihood: Medium - **Impact:** High - **Mitigation:** - Use multiple speech recognition engines - Claude fallback for complex queries - Pattern-based recognition first - Continuous training on user data - **Status:** Not yet encountered

2. Type System Too Complex - Likelihood: Low - **Impact:** Medium - **Mitigation:** - Start with simple types - Add complexity incrementally - Clear error messages - Good documentation - **Status:** Spec complete, ready to implement

3. Performance Slower Than Manual - Likelihood: Low - **Impact:** High - **Mitigation:** - Aggressive caching - Lazy evaluation - Parallel execution

optimization - Performance benchmarks - **Status:** Algorithms designed for performance

Dependency Risks

1. Claude API Rate Limits - **Likelihood:** Medium - **Impact:** Medium - **Mitigation:** - Local intent patterns first - Claude only for complex cases - Caching of common patterns - Retry with backoff - **Status:** Not yet encountered **2. MCP Protocol Changes** - **Likelihood:** Low - **Impact:** Medium - **Mitigation:** - Abstract MCP layer - Version pinning - Monitor MCP releases - Quick adaptation strategy - **Status:** Monitoring

Resource Risks

1. Time Overruns - **Likelihood:** Medium - **Impact:** Medium - **Mitigation:** - Buffer in estimates (12-16 weeks) - Weekly progress reviews - Adjust scope if needed - Prioritize core features - **Status:** Well-planned **2. Scope Creep** - **Likelihood:** Medium - **Impact:** Medium - **Mitigation:** - Clear phase boundaries - Document future features separately - Strict MVP definition - Regular scope review - **Status:** Controlled ---

Metrics & KPIs

Development Metrics

Code Quality: - Test Coverage: Target 90%+ - Type Coverage (mypy): Target 95%+ - Linting (pylint): Target 9.0+ - Cyclomatic Complexity: Max 10 per function **Performance:** - Compilation Time: < 1s for Level 1-3 - Compilation Time: < 5s for Level 4-6 - Execution Overhead: < 10% vs manual - Memory Usage: < 500MB for typical workflows **Voice Interface:** - Intent Recognition Accuracy: Target 90%+ - Speech-to-Text Accuracy: Target 95%+ - Latency (text → DSL): Target < 500ms - Wake Word Detection: Target 98%+

Project Metrics

Documentation: - API Documentation: 100% of public APIs - User Guide Completeness: 100% - Example Coverage: All 6 levels + common patterns - Tutorial Videos: 5+ videos

Testing: - Unit Tests: 200+ tests - Integration Tests: 50+ tests - End-to-End Tests: 20+ workflows - Performance Benchmarks: 10+ scenarios ---

Timeline & Milestones



Overall Schedule

Week	Phase	Milestone
0	Setup	Environment Ready
1-4	Phase 1: Compiler	✓ Compiler Working
5-7	Phase 2: Runtime	✓ Runtime Working
8-10	Phase 3: Voice	✓ Voice Interface Working
11-12	Phase 4: MCP Server	✓ MCP Integration
13	Phase 5: Integration	✓ Claude Code Works
14-16	Phase 6: Production	✓ Production Ready

Detailed Milestones

Milestone 1: Compiler Complete (Week 4) - Date: TBD - Status: Not Started - Deliverables: - ☐ Working lexer - ☐ Working parser - ☐ Working type checker - ☐ Working DAG builder - ☐ CLI tool - ☐ Test suite (90%+ coverage) **Milestone 2: Runtime Complete** (Week 7) - Date: TBD - Status: Not Started - Deliverables: - ☐ Stratification algorithm - ☐ Execution engine - ☐ Resource manager - ☐ Timeout handling - ☐ CLI: `hekat run` **Milestone 3: Voice Interface Complete** (Week 10) - Date: TBD - Status: Not Started - Deliverables: - ☐ Intent recognition (90%+ accuracy) - ☐ DSL translator - ☐ Speech pipeline - ☐ Voice modes - ☐ CLI: `hekat voice` **Milestone 4: MCP Server Complete** (Week 12) - Date: TBD - Status: Not Started - Deliverables: - ☐ MCP server running - ☐ Tools: compile, execute, validate - ☐ Claude Code integration - ☐ Documentation **Milestone 5: Integration Complete** (Week 13) - Date: TBD - Status: Not Started - Deliverables: - ☐ Artifact generator - ☐ .claude/ structure - ☐ Workflows executable - ☐ Integration tests **Milestone 6: Production Ready** (Week 16) - Date: TBD - Status: Not Started - Deliverables: - ☐ Performance optimized - ☐ Security hardened - ☐

Documentation complete - [] Production deployment ---

Decision Log

2025-10-19: Hybrid Algebraic-Graph Approach

Decision: Use hybrid approach combining category theory (frontend) with DAG execution (backend) **Rationale:** - Category theory provides formal correctness - DAG provides runtime performance - Best of both worlds **Alternatives Considered:** - Pure algebraic: Too abstract for debugging - Pure graph: Less formal correctness **Status:** Approved  ---

2025-10-19: Lark Parser Over PLY

Decision: Use Lark parser with EBNF grammar **Rationale:** - Clean EBNF syntax - Better error messages - Easier to maintain - Good performance **Alternatives Considered:** - PLY: More boilerplate - Hand-written: More work - pyparsing: Less performant **Status:** Approved  ---

2025-10-19: Voice-First Design

Decision: Design voice interface as first-class feature, not afterthought **Rationale:** - Accessibility is core requirement - Working parents use case - Natural language is easier - Progressive disclosure **Alternatives Considered:** - CLI-only: Less accessible - Voice as add-on: Harder to integrate **Status:** Approved  ---

Resources & Links

Documentation

Project: - Main README: [hekat/README.md](#) - Implementation Plan:

[hekat/implementation/IMPLEMENTATION-PLAN.md](#) - Progress (this file):

[hekat/docs/PROGRESS.md](#) **Research:** - DSL Research: [hekat/research/DSL-](#)

[RESEARCH.md](#) - Visual Reference: [hekat/research/dsl-visual-reference.md](#) **Design:** -

Formal Spec: `hekat/design/dsl-specification.md` - **Production Spec:**
`hekat/design/DSL-ORCHESTRATION-REFINED.md` - **API Blueprint:** `hekat/design/dsl-api-blueprint.md` **User Guides:** - **Complexity Levels:** `hekat/docs/DSL-COMPLEXITY-LEVELS.md` - **Comprehensive Guide:** `hekat/docs/DSL-ORCHESTRATION-COMPREHENSIVE.md`
- **Quick Reference:** `hekat/docs/DSL-SYMBOLIC-VISUAL-GUIDE.md` - **Voice Interface:**
`hekat/docs/DSL-VERBAL-INTERFACE.md` - **Examples:** `hekat/docs/dsl-examples.md`

External Resources

Linear: - Parent Issue: [CET-172](#) - Team: Ceti-luxor - Project: LUXOR **References:** - Category Theory: Bartosz Milewski's "Category Theory for Programmers" - Graph Theory: CLRS "Introduction to Algorithms" - DSL Design: Martin Fowler's "Domain-Specific Languages" - Claude Code: <https://docs.claude.com/en/docs/claude-code> ---

Team & Contributors

Project Lead: TBD **Developers:** TBD **Reviewers:** TBD **Current Contributors:** - Research & Design: Complete - Implementation: Starting ---

Notes & Comments

2025-10-19

Progress Update: - Completed all research and design documentation - Created comprehensive implementation plan - Organized project structure - Ready to begin Phase 1 (Compiler) **Observations:** - Documentation is extremely thorough (331KB) - Mathematical foundations are solid - Implementation path is clear - Voice interface is well-specified **Next Session:** - Set up development environment - Create Linear sub-issues - Begin Lexer implementation - Establish development workflow **Key Decisions Needed:** - Choose between Lark and PLY parser (recommend Lark) - Decide on testing framework (recommend pytest) -

Determine CI/CD platform (recommend GitHub Actions) ---

Appendix

Complexity Level Reference

Level 1: Basic (5-15K tokens, 2-5 min)

```
api-architect : "design REST API"
```

Level 2: Binary (10-30K tokens, 5-15 min)

```
research -> design -> implement  
frontend || backend
```

Level 3: Parallel Streams (40-100K tokens, 20-45 min)

```
(/deep + /ctx7 || /orch /wflw) : "task"
```

Level 4: Complex (80-150K tokens, 45-90 min)

```
test -> if(pass) deploy : (fix -> retest)
```

Level 5: Workflows (120-250K tokens, 90-180 min)

```
workflow microservice_dev { ... }
```

Level 6: Meta-programming (200K+ tokens, 3+ hr)

```
workflow_generator<T> { ... }
```

Operator Reference

Operator	Symbol	Precedence	Associativity	Example
:	:	5 (highest)	Right	<code>a : "task"</code>
		4	Left	<code>a b</code>
->	->	3	Right	<code>a -> b</code>
+	+	2	Left	<code>a + s</code>
()	()	1 (lowest)	N/A	<code>(a b)</code>

Tech Stack Summary

Language: Python 3.11+ **Parser:** Lark (EBNF) **Graph:** NetworkX **Voice:** SpeechRecognition + Claude **MCP:** TypeScript SDK **API:** FastAPI **Testing:** pytest + hypothesis **CI/CD:** GitHub Actions (recommended) --- **End of Progress Report Next Update:** After Phase 1 Week 1 (Lexer Implementation) **Report Frequency:** Weekly during active development